

Machine Learning in Practice I

Lecture 4

Antal Jakovac, 2025

Data management models, storage, sampling, augmentation, labelling



data models:

- for more complicated data, and if queries are required, storing data in simple files is not enough
- instead: databases with dedicated database handlers
 - ➔ relational (e.g. Sqlite)
 - ➔ nonrelational (e.g. Redis, Mongo)



data models:

- relational
 - tabular logic: main tables, subtables for standardization, index tables for faster query
 - strict schemas
 - query language for data retrieval → SQL (structured query language): declarative, specify the result, not the algorithm
 - **pros**: simple logic, widely used
 - **cons**: abundant schema systems, lots of superficial data, tedious to introduce new features, complicated table structure

data models:

- relational

- tabular logic
faster query
- strict schema
- query language
declarative,
- **pros:** simple
- **cons:** abundant
introduce n

#example sqlite application – built in Python package

```
import sqlite3
```

#create/connect database

```
conn = sqlite3.connect("Data/books.db")
```

```
cur = conn.cursor()
```

#create tables

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS publishers (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL
```

```
)
```

```
""")
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS books (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    title TEXT NOT NULL,  
    author TEXT NOT NULL,  
    year INTEGER,  
    publisher_id INTEGER,  
    FOREIGN KEY (publisher_id) REFERENCES publishers(id)
```

```
)
```

```
""")
```

data models:

- relational

- tabular logic
faster query
- strict schema
- query language
declarative
- **pros:** simple
- **cons:** abundant
introduce n

```
#insert a publisher
```

```
cur.execute("INSERT INTO publishers (name) VALUES (?)", ("George Allen & Unwin",))
```

```
publisher_id = cur.lastrowid
```

```
# Insert a book linked to that publisher
```

```
cur.execute("INSERT INTO books (title, author, year, publisher_id) VALUES  
(?, ?, ?, ?)", ("The Hobbit", "J.R.R. Tolkien", 1937, publisher_id))
```

```
#commit changes
```

```
conn.commit()
```

```
-----
```

```
# Query join
```

```
cur.execute("""
```

```
SELECT b.title, b.author, p.name AS publisher
```

```
FROM books b
```

```
JOIN publishers p ON b.publisher_id = p.id
```

```
""")
```

```
print(cur.fetchall())
```

```
conn.close()
```

```
-----
```

```
#output
```

```
[('The Hobbit', 'J.R.R. Tolkien', 'George Allen & Unwin')]
```



data models:

- NoSQL: no SQL → not only SQL
 - pros: no schemas → flexibility, scalability, treat of unstructured data
 - cons: storage capacity, performance trade-offs, data chaos
 - document stores: data are stored as a JSON (string)
 - query can be a more difficult task
 - examples: mongoDB, CouchDB – create own index tables for effectivity
 - key-value pairs: for fast query (example: RedisDB, DynamoDB)
 - graph databases: concentrate on relations (nodes and edges)
(Examples: Neo4j, Amazon Neptune, ArangoDB)



data models:

- NoSQL: no S

- ➔ pros: no sc

- ➔ cons: stora

- ➔ document s

- query ca

- examples

- ➔ key-value p

- ➔ graph data

(Examples: N

```
#install mongodb-image and pymongo
```

```
docker run -d --name mongodb -p 27017:27017 -v ~/my_data/mongo:/data/db mongo:7.0
```

```
pip install pymongo
```

```
-----
```

```
#runs a service in the background – start/stop it
```

```
docker start/stop mongodb
```

```
-----
```

```
#install mongodb and pymongo
```

```
from pymongo import MongoClient
```

```
# 1. Connect to local MongoDB
```

```
client = MongoClient("mongodb://localhost:27017/")
```

```
# 2. Create (or use existing) database
```

```
db = client["library"]
```

```
# 3. Create collections
```

```
publishers = db["publishers"]
```

```
books = db["books"]
```

```
# 4. Clear old data (for demo purposes)
```

```
publishers.delete_many({})
```

```
books.delete_many({})
```

data

- No

5. Insert publishers

```
pub_allen = publishers.insert_one({"name": "George Allen & Unwin"}).inserted_id
pub_secker = publishers.insert_one({"name": "Secker & Warburg"}).inserted_id
pub_chatto = publishers.insert_one({"name": "Chatto & Windus"}).inserted_id
pub_little = publishers.insert_one({"name": "Little, Brown and Company"}).inserted_id
```

6. Insert books with publisher references

```
books.insert_many([
    {"title": "The Hobbit", "author": "J.R.R. Tolkien", "year": 1937, "publisher_id": pub_allen},
    {"title": "The Fellowship of the Ring", "author": "J.R.R. Tolkien", "year": 1954, "publisher_id": pub_allen},
    {"title": "Nineteen Eighty-Four", "author": "George Orwell", "year": 1949, "publisher_id": pub_secker},
    {"title": "Brave New World", "author": "Aldous Huxley", "year": 1932, "publisher_id": pub_chatto},
    {"title": "The Catcher in the Rye", "author": "J.D. Salinger", "year": 1951, "publisher_id": pub_little},
])
```

7. Query: find all books and join publisher info manually

```
for book in books.find():
    publisher = publishers.find_one({"_id": book["publisher_id"]})
    print(f"{book['title']} by {book['author']} "
          f"was published by {publisher['name']} in {book['year']}")
```



data storage:

- data warehouse:
 - stores historical data → structured, cleaned, integrated
 - optimized for analysis, reporting → few but very large, complex queries
 - ETL (extract, transform, load)
 - OLAP (online analytic processing)
 - examples: historical sales data, stock price data
 - solutions: Amazon Redshift, Google BigQuery

data storage:

- database:
 - stores current, operational data.
 - used for day-to-day transactions (insert, update, delete).
 - optimized for fast read/write, many small simple queries
 - example: an e-commerce database storing users, orders, payments
 - OLTP (Online Transaction Processing)
 - solutions: MySQL, PostgreSQL, Oracle DB, MongoDB, DynamoDB



data storage:

- data lake:
 - stores data in raw form → future-proof
 - application goal not needed to be specify
 - cheap and scalable storage (HDFS, Amazon S3, Azure Data Lake Storage, Google Cloud Storage)
 - schema-on-read solutions (ELT, extract, load, transform) → processing engines (Kafka, TensorFlow, Athena)
 - example: log storage, IoT data storage



data sampling: collecting all data usually not feasible

- nonprobability sampling → focus on data collection; prone to selection bias
 - ➔ convenience sampling: take all what is available
 - ➔ snowball sampling: start with a small set, and follow its descendants
 - ➔ judgement sampling: experts say what to collect
 - ➔ quota sampling: collect the same number of data from different areas (like age groups)
 - ➔ example of bias: psychology experiments (university students), sentiment analysis (wordy people), medical data (unsuccessful treatments)

data sampling:

- probability sampling → needs a data model → biased?
 - stratified sampling: collect the same amount of data per class
 - weighted sampling: take into account data scaled with probability
 - in choice: class A 25%, class B 75% → consider A samples 3 times
 - in training: give weight for underrepresented classes (see later)
 - reservoir sampling: keep the first k sample, and replace it randomly
 - importance sampling

$$E_P[x] = \sum_x x P(x) = \sum_x x P \frac{P(x)}{Q(x)} Q(x) = E_Q \left[x P \frac{P(x)}{Q(x)} \right]$$



data augmentation:

- if there are not enough data, or not representative enough, we can transform data to generate new ones (symmetries)
 - ➔ rotation, scaling
 - ➔ perturbation, noising
 - ➔ texture or style change
- data generation: simulations in artificial environment (e.g. self-driving cars)



labelling:

- for classification we should collect examples in different classes
- problems:
 - how unambiguous are the labels? → multi-label classes
 - how precise are the labels? → no database is 100% correct!
 - what to do with missing labels? → label functions, semi-supervised, etc.
- hand labelled data, naturally labelled data (context is given), automatically generated data

Monitoring and evaluation

How can we assess the success of the ML results?

- not unique! depends on the problem on hand
 - example: percentage of correctly classified images
 - if the probability of belonging to class A is 99%, then the simple method saying that *all* elements belong to A is 99% accurate!
 - clearly not good, if the task is to find the 1% not-A class!



- baselines: how accurate are simple methods and other approaches?
 - random baseline: choose class randomly → $1/N$ probability for balanced
 - zero rule: all belonging to one class → high probability for imbalanced
 - simple heuristic → use a simple proxy (feature)
 - human experts → how well do humans perform?
 - other ML methods

- confusion matrix

- correct classes vs. estimated classes
- works for multi-class evaluation
- in probabilistic sense joint probability: $C_{ij} = P(\text{predicted}_i, \text{actual}_j)$

	Actual Yes	Actual No	Predicted Total
Predicted Yes	TP=0.510	FP=0.038	PP=0.548
Predicted No	FN=0.032	TN=0.420	PN=0.452
Actual Total	AP=0.542	AN=0.478	1.0

- correctness measures for 2-class evaluation

- *accuracy*: prob. of choosing the correct class: $(TP+TN)$

- *precision*: conditional for Predicted Yes: $P(\text{actual}_p | \text{predicted}_p) = \frac{TP}{PP}$

- *recall*: conditional for Actual Yes: $P(\text{predicted}_p | \text{actual}_p) = \frac{TP}{AP}$

- *type I error*: probability of false alarm: $P(\text{predicted}_p | \text{actual}_N) = \frac{FP}{AN}$

- *type II error*: probability of missing signal: $P(\text{predicted}_N | \text{actual}_p) = \frac{FN}{AP}$



- controlled case → measure does not matter
 - probability of class A or B is 50%, predict symmetrically
 - accuracy=98%
 - for class A: precision=98%, recall=98%, type II error=2%
 - for class B: precision=98%, recall=98%, type I error=2%

	Actual Yes	Actual No	Predicted Total
Predicted Yes	TP=0.49	FP=0.01	PP=0.5
Predicted No	FN=0.01	TN=0.49	PN=0.5
Actual Total	AP=0.5	AN=0.5	1.0

- extreme cases
 - classify everything as class A, but probability of class A is 99%
 - accuracy=99%
 - for class A: precision=99%, recall=100%, type II error=0%
 - for class B: precision=?, recall=0%, type I error=100%

	Actual Yes	Actual No	Predicted Total
Predicted Yes	TP=0.99	FP=0.01	PP=1.0
Predicted No	FN=0.0	TN=0.0	PN=0.0
Actual Total	AP=0.99	AN=0.01	1.0

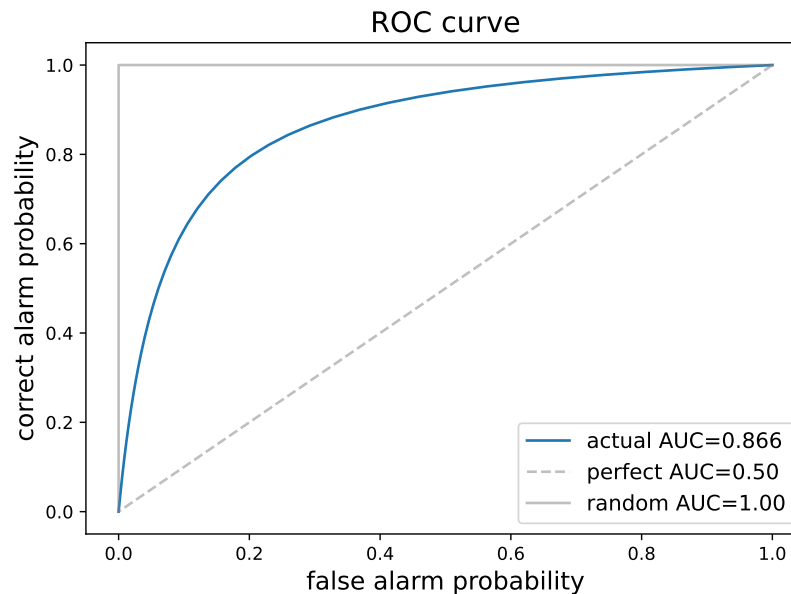
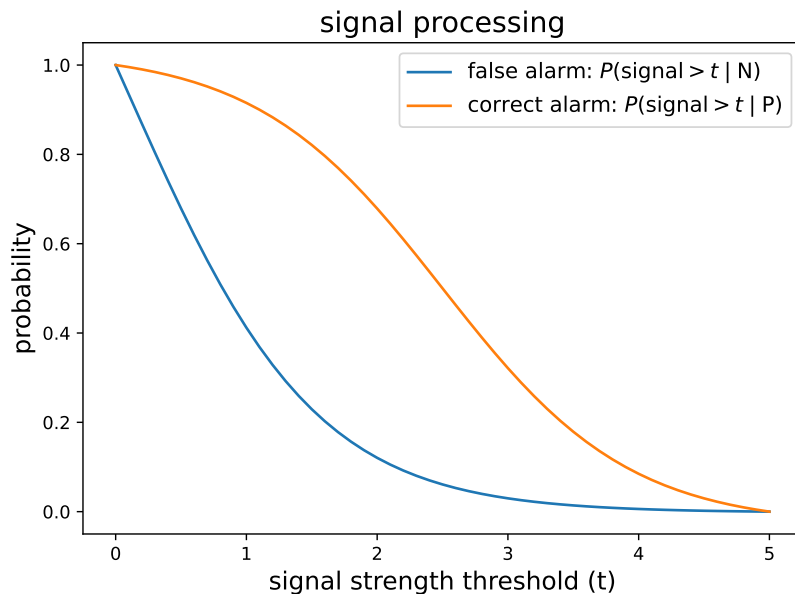
- extreme cases
 - classify everything randomly, but probability of class A is 98%
 - accuracy=50%
 - for class A: precision=98%, recall=50%, type II error=50%
 - for class B: precision=2%, recall=50%, type I error=50%

	Actual Yes	Actual No	Predicted Total
Predicted Yes	TP=0.49	FP=0.01	PP=0.5
Predicted No	FN=0.49	TN=0.01	PN=0.5
Actual Total	AP=0.98	AN=0.02	1.0



- effect of a threshold on the prediction
 - in lot of cases Yes/No decision comes from comparing a signal with a threshold → e.g. smoke detector signal vs. danger level
 - ROC (receiver operating characteristics)
 - $P(\text{predicted}_P | \text{actual}_P)$ vs. $P(\text{predicted}_P | \text{actual}_N)$
 - recall vs. type I error
 - correct alarm probability vs. false alarm probability
 - AUC: area under curve

- effect of a threshold on the prediction



END OF LECTURE 4